

I need a Python function called "load_event_log(filepath: str) → pd.DataFrame" that uses pm4py to load an XES event log file and converts it to a pandas DataFrame. The resulting DataFrame must contain at least these columns: 'case:concept:name' (case ID), 'concept:name' (activity name), 'time:timestamp' (event timestamp). The function should sort events by case ID and timestamp. Use pm4py.read_xes() for loading

👋 LISA Flash

Here is the Python function that fulfills your requirements. It uses `pm4py` to read the XES file, converts it to a pandas DataFrame, ensures the required columns exist (handling potential namespace variations if necessary), and sorts the data as requested.

python

⌵ Свернуть ⌵ Запустить ⌵ Копировать

```
import pm4py
import pandas as pd

def load_event_log(filepath: str) -> pd.DataFrame:
    """
    Loads an XES event log file using pm4py and converts it to a sorted pandas
    DataFrame.

    Parameters:
        filepath (str): Path to the .xes file.

    Returns:
        pd.DataFrame: A DataFrame containing 'case:concept:name', 'concept:name',
        and 'time:timestamp'.
    """
    # Load the XES log
    log = pm4py.read_xes(filepath)

    # Convert to pandas DataFrame
    df = pm4py.convert_to_dataframe(log)

    # Ensure standard column names are present.
    # pm4py often normalizes these, but we ensure they match the request exactly.
    # If the log uses different namespaces (e.g., 'concept:name' vs 'name'), pm4py
    usually
    # handles this during conversion, but we verify the specific keys requested.

    required_columns = ['case:concept:name', 'concept:name', 'time:timestamp']

    # Check if column exists; if not, attempt to find common alternatives used by
```

```

# Check if columns exist; if not, attempt to find common alternatives used by
pm4py
# Note: pm4py's convert_to_dataframe usually maps them directly to these keys.
# If the source log uses different attribute names, you might need to rename
them here.
# For standard XES logs loaded via pm4py, the keys below are typically correct.

# Sort by case ID and then by timestamp
# We assume 'time:timestamp' is in datetime format. If it's a string,
pd.to_datetime may be needed.
df = df.sort_values(by=['case:concept:name', 'time:timestamp'])

return df

```

The DataFrame contains a column 'lifecycle:transition' with values like 'COMPLETE', 'START', 'SCHEDULE'. Please update the "load_event_log" function to filter the DataFrame and keep only rows where 'lifecycle:transition' equals 'COMPLETE'. If the column does not exist in the DataFrame, skip the filtering step. Apply this filter before sorting

👋 LISA Flash

Here is the updated Python function. It now includes a check for the 'lifecycle:transition' column and filters for 'COMPLETE' events before sorting the data.

python 🔍 Свернуть ⚙️ Запустить 📋 Копировать

```

import pm4py
import pandas as pd

def load_event_log(filepath: str) -> pd.DataFrame:
    """
    Loads an XES event log file using pm4py, converts it to a pandas DataFrame,
    filters for 'COMPLETE' lifecycle transitions if available, and sorts the
    result.

    Parameters:
        filepath (str): Path to the .xes file.

    Returns:
        pd.DataFrame: A DataFrame containing 'case:concept:name', 'concept:name',
            'time:timestamp', and optionally other columns, sorted by
            case ID and timestamp.
    """
    # Load the XES log

```



```

log = pm4py.read_xes(filepath)

# Convert to pandas DataFrame
df = pm4py.convert_to_dataframe(log)

# Filter for 'COMPLETE' lifecycle transition if the column exists
if 'lifecycle:transition' in df.columns:
    df = df[df['lifecycle:transition'] == 'COMPLETE'].reset_index(drop=True)

# Ensure required columns exist (optional safety check, though pm4py usually
provides them)
required_cols = ['case:concept:name', 'concept:name', 'time:timestamp']
if not all(col in df.columns for col in required_cols):
    raise ValueError(f"DataFrame is missing required columns: {required_cols}")

# Sort by case ID and then by timestamp
df = df.sort_values(by=['case:concept:name', 'time:timestamp'])

return df

```

I need a Python function called "create_prefixes(df: pd.DataFrame) → pd.DataFrame" that takes an event log DataFrame and generates all possible prefixes for each case. For each case, a prefix of length k contains the first k activities. The function should return a new DataFrame where each row represents one prefix, with columns: 'case_id', 'prefix_activities' (list of activity names), 'prefix_length' (integer k), 'next_activity' (the activity after the prefix, or None if it's the last event), 'remaining_time' (total remaining time in seconds from the last event of the prefix to the end of the case, as a float). Cases with only one event should be excluded.

Note: the input DataFrame has columns 'case:concept:name' for case ID, 'concept:name' for activity name, and 'time:timestamp' for timestamps. Use these exact column names.

👉 LISA Flash

Here is the Python function `create_prefixes` that generates all possible prefixes for each case in the event log, calculates the remaining time, and handles the specific requirements you outlined.

python

🔗 Свернуть Запустить Копировать

```

import pandas as pd
import numpy as np

def create_prefixes(df: pd.DataFrame) -> pd.DataFrame:
    """
    Generates all possible prefixes for each case in an event log DataFrame.

```

Parameters:

`df (pd.DataFrame)`: Input DataFrame with columns 'case:concept:name', 'concept:name', and 'time:timestamp'.
Must be sorted by case ID and timestamp.

Returns:

`pd.DataFrame`: A DataFrame containing one row per prefix with columns: 'case_id', 'prefix_activities', 'prefix_length', 'next_activity', 'remaining_time'.

"""

Ensure the input is sorted correctly to maintain sequence integrity

if not df.empty:

df = df.sort_values(by=['case:concept:name',

'time:timestamp']).reset_index(drop=True)

results = []

Group by case ID

grouped = df.groupby('case:concept:name')

for case_id, group **in** grouped:

Convert group to list of activities and timestamps for easier indexing

activities = group['concept:name'].tolist()

timestamps = group['time:timestamp'].tolist()

n_events = len(activities)

Exclude cases with only one event as per requirements

if n_events <= 1:

continue

Iterate through possible prefix lengths (k from 1 to n_events - 1)

A prefix of length k includes events at indices 0 to k-1

for k **in** range(1, n_events):

prefix_acts = activities[:k]

prefix_len = k

Determine the next activity (the one immediately following the
prefix)

next_act = activities[k]


```

# Calculate remaining time
# The prefix ends at index k-1. The case ends at index n_events-1.
# We need the time difference between the last event of the full case
# and the last event of the current prefix.

prefix_last_ts = timestamps[k-1]
case_last_ts = timestamps[-1]

# Ensure timestamps are numeric or convert to seconds if they are
datetime objects
if isinstance(prefix_last_ts, pd.Timestamp) or isinstance(case_last_ts,
pd.Timestamp):
    diff_seconds = (case_last_ts - prefix_last_ts).total_seconds()
else:
    # Assuming raw float/int seconds if not Timestamp
    diff_seconds = float(case_last_ts) - float(prefix_last_ts)

results.append({
    'case_id': case_id,
    'prefix_activities': prefix_acts,
    'prefix_length': prefix_len,
    'next_activity': next_act,
    'remaining_time': diff_seconds
})

if not results:
    return pd.DataFrame(columns=['case_id', 'prefix_activities',
'prefix_length', 'next_activity', 'remaining_time'])

return pd.DataFrame(results)

```

I need a Python class called "MostFrequentClassBaseline" with two methods. The "fit(df: pd.DataFrame)" method takes a DataFrame with a column 'next_activity' and finds the most frequently occurring activity in that column, storing it as "self.most_frequent". The "predict(df: pd.DataFrame) → list" method returns a list of predictions where every prediction is "self.most_frequent", with length equal to the number of rows in df

LISA Flash

Here is the Python class `MostFrequentClassBaseline` implementing the logic you described:

python

⌕ Свернуть Запустить Копировать

```

import pandas as pd
from typing import List, Any

class MostFrequentClassBaseline:
    def __init__(self):
        self.most_frequent = None

    def fit(self, df: pd.DataFrame) -> 'MostFrequentClassBaseline':
        """
        Fits the model by finding the most frequently occurring activity
        in the 'next_activity' column.

        Parameters:
            df (pd.DataFrame): DataFrame containing the 'next_activity' column.

        Returns:
            MostFrequentClassBaseline: The fitted instance.
        """
        if 'next_activity' not in df.columns:
            raise ValueError("DataFrame must contain a 'next_activity' column.")

        # Count values and get the most common one
        # mode() returns a Series; we take the first value if there are ties
        mode_result = df['next_activity'].mode()

        if len(mode_result) == 0:
            # Handle case where column is empty or all NaN
            self.most_frequent = None
        else:
            self.most_frequent = mode_result.iloc[0]

        return self

    def predict(self, df: pd.DataFrame) -> List[Any]:
        """
        Predicts the most frequent activity for every row in the input DataFrame.

        Parameters:
            df (pd.DataFrame): DataFrame to generate predictions for.

        Returns:

```

List[Any]: A list of predictions, each being the stored most frequent activity.

The length of the list equals the number of rows in df.

```
"""
if self.most_frequent is None:
    raise RuntimeError("Model has not been fitted yet. Call fit() first.")

return [self.most_frequent] * len(df)
```

I need a Python class called "MeanRemainingTimeBaseline" with two methods. The "fit(df: pd.DataFrame)" method takes a DataFrame with a column "remaining_time" containing float values in seconds, and computes the mean value, storing it as "self.mean_time". The "predict(df: pd.DataFrame) → list" method returns a list where every value equals "self.mean_time", with length equal to the number of rows in df

👋 LISA Flash

Here is the Python class `MeanRemainingTimeBaseline` implementing your requirements:

python

⌕ Свернуть Запустить Копировать

```
import pandas as pd
from typing import List, Union

class MeanRemainingTimeBaseline:
    def __init__(self):
        self.mean_time: Union[float, None] = None

    def fit(self, df: pd.DataFrame) -> 'MeanRemainingTimeBaseline':
        """
        Fits the model by computing the mean of the 'remaining_time' column.

        Parameters:
            df (pd.DataFrame): DataFrame containing the 'remaining_time' column
            with float values.

        Returns:
            MeanRemainingTimeBaseline: The fitted instance.
        """
        if 'remaining_time' not in df.columns:
            raise ValueError("DataFrame must contain a 'remaining_time' column.")

        # Calculate mean, handling potential NaNs by default behavior of pandas
        mean()
```



```

# If the column is empty or all NaN, this will return NaN, which is stored.
self.mean_time = df['remaining_time'].mean()

return self

def predict(self, df: pd.DataFrame) -> List[float]:
    """
    Predicts the mean remaining time for every row in the input DataFrame.

    Parameters:
        df (pd.DataFrame): DataFrame to generate predictions for.

    Returns:
        List[float]: A list where every value equals self.mean_time.
                     The length of the list equals the number of rows in df.
    """
    if self.mean_time is None:
        raise RuntimeError("Model has not been fitted yet. Call fit() first.")

    return [self.mean_time] * len(df)

```

I need a Python module with three evaluation functions. First: "accuracy(y_true: list, y_pred: list) → float" that computes classification accuracy. Second: "mae(y_true: list, y_pred: list) → float" that computes Mean Absolute Error between two lists of floats. Third: "evaluate_by_prefix_length(df: pd.DataFrame, y_pred_activity: list, y_pred_time: list) → pd.DataFrame" that groups predictions by the column 'prefix_length' in df and computes accuracy for activity prediction and MAE for remaining time prediction for each prefix length, returning a DataFrame with columns 'prefix_length', 'accuracy', 'mae'

👋 LISA Flash

Here is the Python module containing the three evaluation functions you requested.

python

🔍 Свернуть ⏏ Запустить 📋 Копировать

```

import pandas as pd
from typing import List, Any

def accuracy(y_true: list, y_pred: list) -> float:
    """
    Computes classification accuracy between two lists.

```

Parameters:

y_true (list): List of true labels

`y_true (list):` List of true labels.

`y_pred (list):` List of predicted labels.

Returns:

`float:` Accuracy score (number of correct predictions / total predictions).

Returns `0.0` if lists are empty.

```
"""
if len(y_true) != len(y_pred):
    raise ValueError("y_true and y_pred must have the same length.")
```

```
if not y_true:
    return 0.0
```

```
correct = sum(1 for t, p in zip(y_true, y_pred) if t == p)
return correct / len(y_true)
```

```
def mae(y_true: list, y_pred: list) -> float:
```

```
"""
```

Computes Mean Absolute Error between two lists of floats.

Parameters:

`y_true (list):` List of true float values.

`y_pred (list):` List of predicted float values.

Returns:

`float:` Mean Absolute Error.

Returns `0.0` if lists are empty.

```
"""
if len(y_true) != len(y_pred):
    raise ValueError("y_true and y_pred must have the same length.")
```

```
if not y_true:
    return 0.0
```

```
try:
    errors = [abs(float(t) - float(p)) for t, p in zip(y_true, y_pred)]
    return sum(errors) / len(errors)
```

```
except (TypeError, ValueError):
    raise ValueError("y_true and y_pred must contain numeric values convertible to float.")
```

```

def evaluate_by_prefix_length(df: pd.DataFrame, y_pred_activity: list, y_pred_time:
list) -> pd.DataFrame:
    """
    Evaluates activity prediction accuracy and remaining time MAE grouped by
    'prefix_length'.

    Parameters:
        df (pd.DataFrame): DataFrame containing at least 'prefix_length',
        'next_activity',
                                and 'remaining_time' columns.
        y_pred_activity (list): List of predicted next activities corresponding to
        rows in df.
        y_pred_time (list): List of predicted remaining times corresponding to rows
        in df.

    Returns:
        pd.DataFrame: A DataFrame with columns 'prefix_length', 'accuracy', and
        'mae'.
    """
    if len(df) != len(y_pred_activity) or len(df) != len(y_pred_time):
        raise ValueError("Input lists must have the same length as the number of
        rows in df.")

    # Add predictions to the dataframe for easier grouping
    eval_df = df.copy()
    eval_df['pred_activity'] = y_pred_activity
    eval_df['pred_time'] = y_pred_time

    results = []

    # Group by prefix_length
    grouped = eval_df.groupby('prefix_length')

    for prefix_len, group in grouped:
        # Extract ground truth and predictions for this group
        true_acts = group['next_activity'].tolist()
        pred_acts = group['pred_activity'].tolist()
        true_times = group['remaining_time'].tolist()

```



```

pred_times = group['pred_time'].tolist()

# Calculate metrics using the helper functions
acc_score = accuracy(true_acts, pred_acts)
mae_score = mae(true_times, pred_times)

results.append({
    'prefix_length': prefix_len,
    'accuracy': acc_score,
    'mae': mae_score
})

return

```

```

pd.DataFrame(results).sort_values(by='prefix_length').reset_index(drop=True)

```

When I run `load_event_log` on the BPI Challenge 2017 dataset, the function returns an empty DataFrame with 0 events. The reason is that the `lifecycle:transition` filter for 'COMPLETE' (case-sensitive) removes all events from this dataset, because BPI Challenge 2017 uses a different capitalization.

Please update the `load_event_log` function with the following fallback logic:

Apply the existing filter: keep rows where `lifecycle:transition == 'COMPLETE'`

If the result is empty, try a case-insensitive filter: keep rows where `lifecycle:transition.str.upper() == 'COMPLETE'`

If still empty, skip the `lifecycle:transition` filter entirely and use all events

Keep all other logic unchanged (sorting, required columns check, return type)

👋 LISA Flash

Here is the updated `load_event_log` function with the requested fallback logic to handle different capitalizations of the 'COMPLETE' lifecycle transition in the BPI Challenge 2017 dataset.

python

🔗 Свернуть Запустить Копировать

```

import pm4py
import pandas as pd

```

```

def load_event_log(filepath: str) -> pd.DataFrame:
    """

```

Loads an XES event log file using `pm4py`, converts it to a pandas DataFrame, applies a robust filter for 'COMPLETE' lifecycle transitions, and sorts the result.

Parameters:

`filepath (str)`: Path to the .xes file.

Returns:

pd.DataFrame: A DataFrame containing 'case:concept:name', 'concept:name', 'time:timestamp', sorted by case ID and timestamp.

"""

Load the XES log

log = pm4py.read_xes(filepath)

Convert to pandas DataFrame

df = pm4py.convert_to_dataframe(log)

if 'lifecycle:transition' in df.columns:

Step 1: Try exact match first

filtered_df = df[df['lifecycle:transition'] == 'COMPLETE']

if filtered_df.empty:

Step 2: If empty, try case-insensitive match

Ensure the column is treated as string for .str accessor

filtered_df = df[df['lifecycle:transition'].astype(str).str.upper() ==

'COMPLETE']

if filtered_df.empty:

Step 3: If still empty, skip the filter entirely

filtered_df = df

else:

Exact match worked, use it

filtered_df = filtered_df

df = filtered_df

Ensure required columns exist

required_cols = ['case:concept:name', 'concept:name', 'time:timestamp']

missing_cols = [col for col in required_cols if col not in df.columns]

if missing_cols:

raise ValueError(f"DataFrame is missing required columns: {missing_cols}")

Sort by case ID and then by timestamp

df = df.sort_values(by=['case:concept:name',

'time:timestamp']).reset_index(drop=True)

return df

